

Project: Prototype 3 - Quality Attributes, Part 1

Team

Name

1e

Team members

- Nicolas Felipe Arciniegas Lizarazo
- Karen Lorena Guzman Del Rio
- Juan David Chacon Munoz
- Adrian Yebid Rincon
- Pablo Felipe Sandoval Menjura
- Julio Cesar Albadan Sarmiento

Software System

Name

FitBeat

Logo



Description

FitBeat is a distributed fitness platform that synchronizes workout sessions with Spotify playback. A user can sign up or log in, connect a Spotify account, configure music preferences, and start a training session. During the session, playback can be controlled in real time, while business events are emitted and consumed asynchronously for achievements, analytics, and notifications.

The third prototype redesigns the previous architecture by separating the public edge from the API orchestration responsibilities. Traefik is now modeled as the reverse proxy at the system boundary, KrakenD remains as the API Gateway in the orchestration tier, and backend services are isolated through segmented Docker networks and service-owned persistence.

Functional completeness

The team-defined functional scope for this prototype includes:

- User registration, login, JWT session management, and Spotify OAuth connection.
- Music preference management for training personalization.
- Training session creation and real-time playback control through HTTP and WebSocket.
- Event emission from training activity to asynchronous consumers.
- Achievement evaluation and progress retrieval.
- Notification generation and persistence from business events.
- Secure external access through HTTPS, reverse proxy routing, API gateway contracts, network segmentation, and service-to-service shared-secret validation.

1. Component-and-Connector Structure

The diagram illustrates a microservices architecture for a music application. It is divided into two main sections: a 'Conventions' legend and a detailed system architecture.

Conventions Legend:

- Data Base:** Represented by a red cylinder icon.
- Logic Component:** Represented by a blue rounded rectangle with two small circles on the left.
- Presentation Component:** Represented by a green rounded rectangle with two small circles on the left.
- External Component:** Represented by a black rounded rectangle with two small circles on the left.
- Servicio:** Represented by a blue circle with a central dot and four lines.
- DB connector:** Represented by a vertical line.
- AMQP connector:** Represented by a red line.
- HTTP connector:** Represented by a blue line.
- HTTPS connector:** Represented by an orange line.
- Network segment:** Represented by a dashed line.

System Architecture:

The architecture is enclosed in a dashed box, indicating a network segment. It features the following components and connections:

- External Services:**
 - spotify_api:** An external logic component that connects to **fb_auth_ms** via HTTP/REST and to the **SMTP Gmail server** via SMTP.
 - SMTP Gmail server:** An external logic component that connects to the **fb_auth_db** via SMTP.
- Facebook Services (fb_*):**
 - fb_fe (Facebook Frontend):** A presentation component that connects to **fb_cli** and **fb_reverse_pr oxy** via HTTPS / TLS 1.2+.
 - fb_cli (Facebook Client Library):** A presentation component that connects to **fb_reverse_pr oxy** via HTTPS / TLS 1.2+.
 - fb_reverse_pr oxy:** A logic component that connects to **fb_api_gateway** via HTTP/REST.
 - fb_api_gateway:** A logic component that acts as a central hub, connecting to **fb_auth_ms**, **fb_achieve_ms**, **fb_notification_ms**, **fb_music_ms**, and **fb_event_ms** via various protocols (HTTP/REST, AMQP, HTTP-REST/WS).
 - fb_be_bkr (Facebook Backend Business Kit):** A logic component that connects to **fb_event_ms** via AMQP.
- Microservices (fb_*_ms):**
 - fb_auth_ms:** Connects to **fb_auth_db** via a DB connector and receives data from **spotify_api** and **SMTP Gmail server** via HTTP/REST.
 - fb_achieve_ms:** Connects to **fb_auth_ms** via HTTP/REST and to **fb_achieve_db** via a DB connector. It also acts as a consumer for an AMQP message from **fb_notification_ms**.
 - fb_notification_ms:** Connects to **fb_achieve_ms** via AMQP (acting as a producer) and to **fb_notification_db** via a DB connector. It also acts as a consumer for an AMQP message from **fb_music_ms**.
 - fb_music_ms:** Connects to **fb_notification_ms** via HTTP-REST/WS (acting as a producer) and to **fb_music_db** via a DB connector.
 - fb_event_ms:** Connects to **fb_api_gateway** via AMQP (acting as a producer/consumer) and to **fb_event_db** via a DB connector.
- Databases (fb_*_db):**
 - fb_auth_db:** Connected to **fb_auth_ms** via a DB connector.
 - fb_achieve_db:** Connected to **fb_achieve_ms** via a DB connector.
 - fb_notification_db:** Connected to **fb_notification_ms** via a DB connector.
 - fb_music_db:** Connected to **fb_music_ms** via a DB connector.
 - fb_event_db:** Connected to **fb_event_ms** via a DB connector.

Presentation components

- ## Edge and orchestration components

- 3 / 19

- **rabbitmq** (**fb_rabbitmq**): asynchronous message broker used for event publication and consumption between backend services.

Logic-type components

- **user-service** (**component_a**, Python/FastAPI): manages users, local authentication, JWTs, Spotify OAuth, and internal contact/token endpoints.
- **music-service** (**music_service**, Go): manages training sessions, Spotify playback orchestration, WebSocket player commands, and event publication.
- **achievements-service** (**achievements_service**, .NET 8): evaluates achievement rules, stores badge progress, and consumes training-related events.
- **notification-service** (**notification_service**, TypeScript/Node.js): consumes events, obtains user contact data through authorized S2S calls, persists notifications, and dispatches email messages.
- **event-processor** (Java/Spring Boot): consumes training events and computes derived metrics and statistics.

Data components

- **fb_users_db** (PostgreSQL): user, credential, token, and preference data.
- **fb_music_db** (CouchDB): training session and music-session documents.
- **fb_achievements_db** (PostgreSQL): achievement catalog, progress, and processed inbound events.
- **fb_notification_db** (PostgreSQL): notification records and delivery information.
- **fb_event_db** (PostgreSQL): processed event records and derived metrics.

External systems

- Spotify API: external provider used for OAuth, playback, current-track information, and music control.
- Email provider: external service used by the notification service to send messages.

Main relations and connectors

Connector	Producer	Consumer	Description
HTTPS/HTTP request-reply	Web, SSR, CLI clients	Traefik	External clients access the system through the reverse proxy. HTTPS is used for secure-channel scenarios.
HTTP reverse proxy	Traefik	KrakenD	Traefik forwards API path prefixes to KrakenD and hides backend topology.
REST/HTTP	KrakenD	Backend services	KrakenD routes public API contracts to user-service , music-service , achievements-service , and notification-service .
WebSocket	Client player UI/CLI	music-service through gateway path	Real-time playback commands are sent during an active training session.

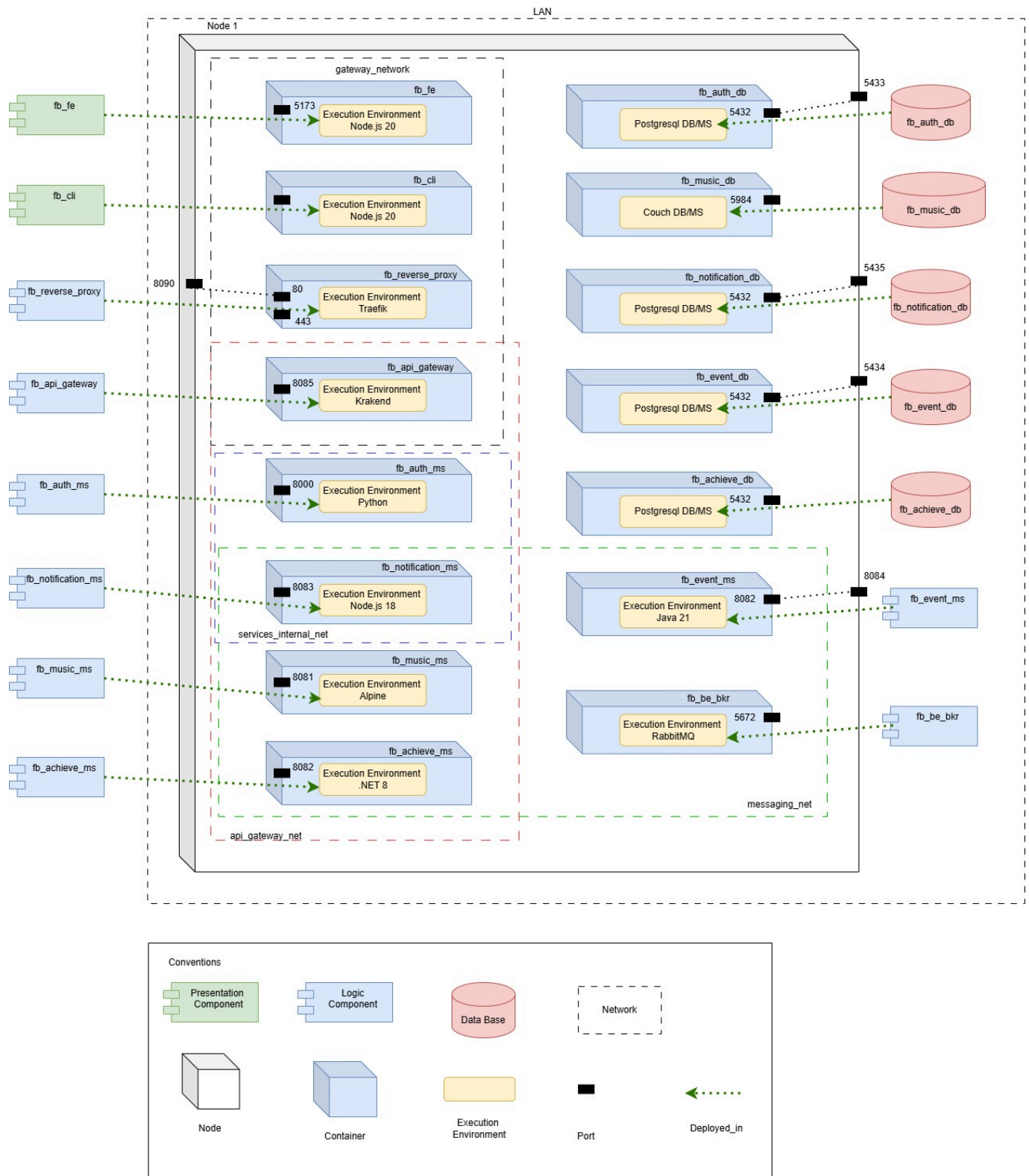
Connector	Producer	Consumer	Description
AMQP	music-service and other producers	RabbitMQ consumers	Business events are published to queues/exchanges for asynchronous processing.
DB connectors	Microservices	Owned databases	Each service accesses only its own persistence component.
HTTPS external API	user-service , music-service	Spotify API	OAuth, token refresh, and playback interactions.
Internal HTTP + X-Internal-Token	notification-service	user-service	Authorized service-to-service access to internal contact data.

Description of architectural styles and patterns used

- **Microservices style:** FitBeat is decomposed into independently deployable backend services aligned with business capabilities: identity, workout/playback, achievements, notifications, and event processing.
- **Client-server style:** Frontends act as clients and backend components expose HTTP/WebSocket interfaces through controlled entry points.
- **Distributed architecture:** Components run in separate containers and communicate through network connectors, which introduces latency and partial-failure concerns handled through explicit contracts and asynchronous messaging.
- **Event-driven collaboration:** Training and business events are delivered through RabbitMQ so producers and consumers do not depend on each other's availability.
- **API Gateway pattern:** KrakenD provides a single API facade and exposes only whitelisted endpoints.
- **Reverse Proxy pattern:** Traefik protects the edge, terminates TLS, applies routing policies, and prevents clients from seeing internal service addresses.
- **Broker pattern:** RabbitMQ decouples fast producers from asynchronous consumers.
- **Database-per-service pattern:** Each service owns its persistence boundary and avoids shared database coupling.
- **Secure Channel pattern:** HTTPS/TLS protects external requests that may contain credentials or tokens.
- **Secret Token pattern:** Internal HTTP endpoints are protected with a shared **X-Internal-Token** header.

2. Deployment Structure

Deployment View



Description of architectural elements and relations

Deployment environment

The prototype is deployed with Docker Compose on a single host. Each major component runs in an independent container with its own runtime image, environment variables, ports, health checks, and Docker network memberships.

Nodes and containers

- Host node: physical or virtual machine running Docker Engine and Docker Compose.
- `fb_gateway`: Traefik reverse proxy container.
- `fb_api_gateway`: KrakenD API Gateway container.
- `fitbeat_frontend`: React + Vite frontend container, used under the `legacy-csr` profile.
- `fitbeat_frontend_ssr`: Next.js SSR frontend container.
- `fitbeat_cli`: Node.js CLI container.
- `fb_users_ms`, `fb_music_ms`, `fb_achievements_ms`, `fb_notification_ms`, `fb_event_processor`: backend service containers.
- `fb_users_db`, `fb_music_db`, `fb_achievements_db`, `fb_notification_db`, `fb_event_db`: persistence containers.
- `fb_rabbitmq`: AMQP broker and management UI container.

Runtime environments

- Traefik v3.1 for reverse proxy, TLS, and routing.
- KrakenD 2.7 for API gateway contracts and backend forwarding.
- Node.js runtime for CSR frontend, SSR frontend, CLI, and notification service.
- Python 3.11 + Uvicorn/FastAPI for user service.
- Go runtime/binary for music service.
- .NET 8 runtime for achievements service.
- Java 21 runtime for event processor.
- PostgreSQL, CouchDB, and RabbitMQ for infrastructure services.

Networks

- `gateway_network`: DMZ/public access zone containing Traefik, KrakenD, frontends, and CLI.
- `api_gateway_net`: internal API zone connecting KrakenD with backend microservices.
- `users_db_net`, `music_db_net`, `achievements_db_net`, `notification_db_net`, `events_db_net`: internal database networks dedicated to each service and its database.
- `messaging_net`: internal AMQP zone for RabbitMQ and event producers/consumers.
- `services_internal_net`: explicit internal channel between `notification-service` and `user-service`.

Main deployment relations

- External clients -> host ports `8090/443` -> `fb_gateway`.
- `fb_gateway` -> `fb_api_gateway` through `gateway_network`.
- `fb_api_gateway` -> backend microservices through `api_gateway_net`.
- Each microservice -> owned database through its dedicated internal database network.
- Event producers/consumers -> `fb_rabbitmq` through `messaging_net`.
- `fitbeat_frontend_ssr` -> Traefik/KrakenD using `NEXT_PUBLIC_GATEWAY_URL`.

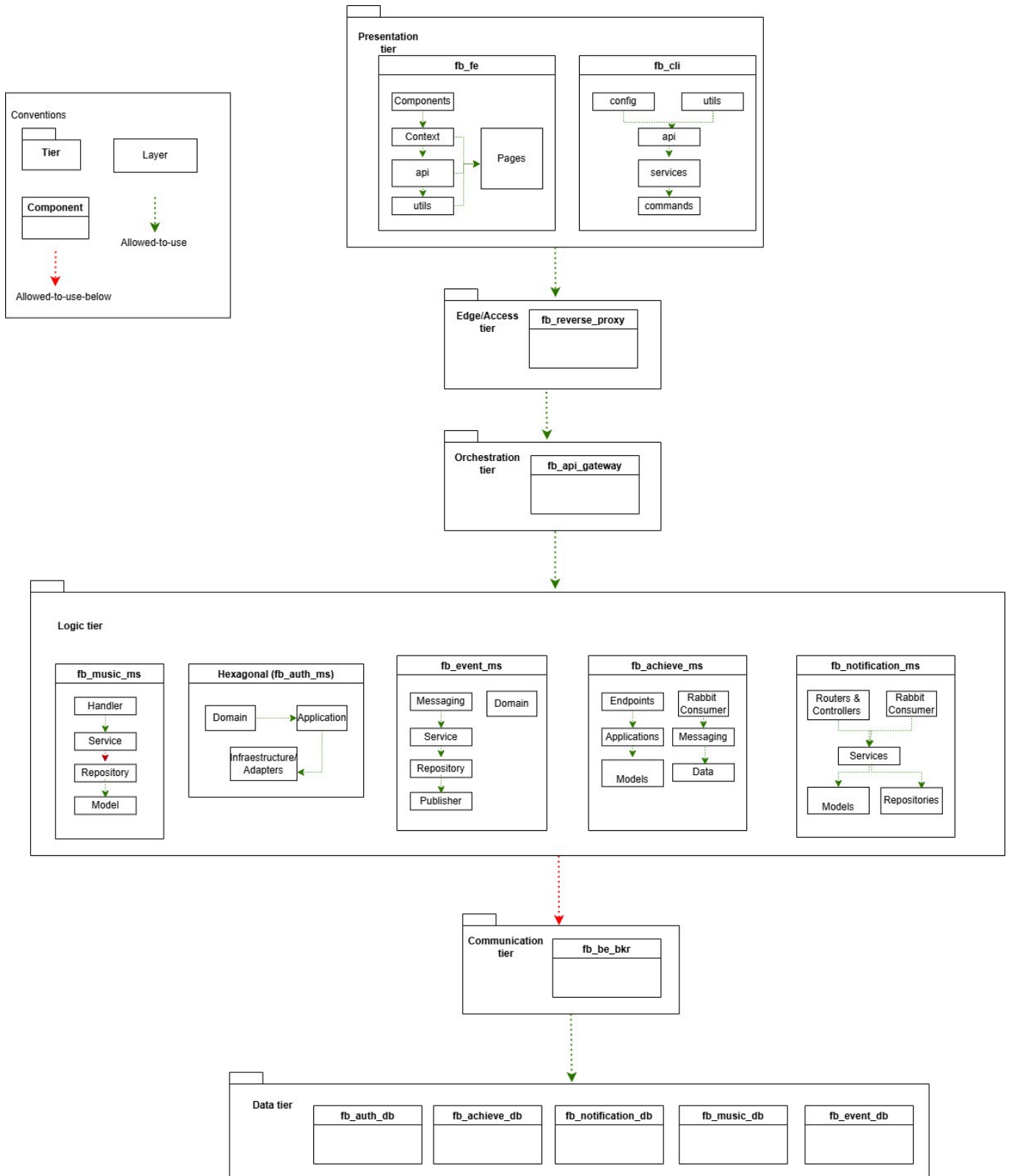
Description of architectural patterns used

- **Container-based deployment:** Every application and infrastructure component is packaged in a container, making runtime dependencies explicit and reproducible.

- **Edge reverse proxy deployment:** Traefik is deployed at the system boundary to receive all external traffic and enforce edge policies.
- **API Gateway deployment:** KrakenD is deployed behind Traefik and is the only bridge between the public gateway network and backend API network.
- **Network Segmentation pattern:** Docker networks create trust zones for public ingress, API routing, data access, messaging, and explicit S2S channels.
- **Dedicated storage pattern:** Each data store is allocated to the domain service that owns it.
- **Defense-in-depth deployment:** TLS, reverse proxy routing, endpoint whitelisting, network isolation, and shared-secret validation operate as independent protection layers.

3. Layered Structure

Layered View



Description of architectural elements and relations

Tier 1 - Client / Presentation

- **frontend/web**, **frontend/web-ssr**, and **frontend/cli**.
- Responsible for user interaction, UI rendering, command-line workflows, session display, and client-side coordination.

Tier 2 - Edge / Access

- `traefik`.
- Responsible for public entry, HTTP/HTTPS exposure, TLS termination, CORS middleware, and first-level routing.

Tier 3 - Orchestration

- `krakend` and `rabbitmq`.
- KrakenD orchestrates synchronous API access by exposing the backend contract and routing requests.
- RabbitMQ orchestrates asynchronous collaboration by decoupling producers and consumers.

Tier 4 - Business Services

- `user-service`, `music-service`, `achievements-service`, `notification-service`, and `event-processor`.
- Responsible for application and domain logic, service-specific validation, event publication/consumption, and integration with external APIs.

Tier 5 - Data and Messaging Infrastructure

- PostgreSQL databases, CouchDB, and RabbitMQ storage/queues.
- Responsible for persistence, event durability, and infrastructure-level data management.

Dependency rule

The allowed-to-use relation flows from upper tiers to lower tiers:

```
Client / Presentation
  -> Edge / Access
    -> Orchestration
      -> Business Services
        -> Data and Messaging Infrastructure
```

Backend services may also use the orchestration tier for asynchronous event publication/consumption. Direct access from clients to business services or databases is not part of the intended architecture.

Logical layers inside selected components

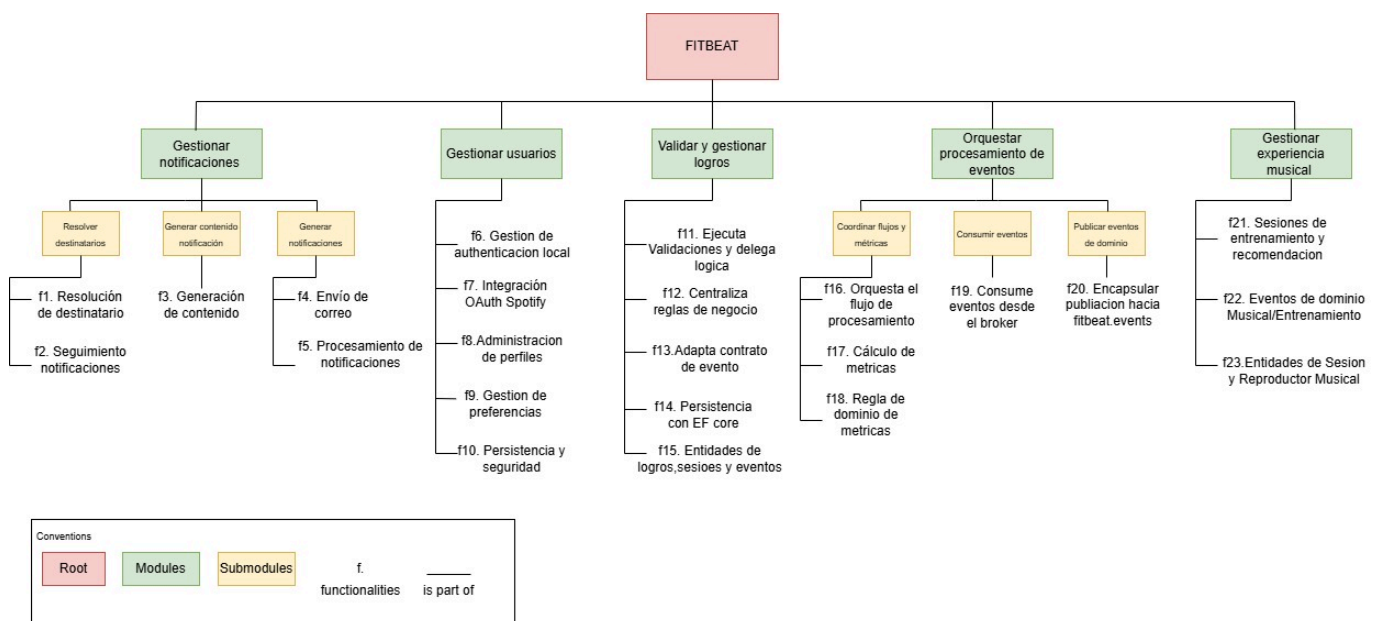
- `frontend/web`: `presentation` -> `state/coordinator` -> `client integration` -> `external interfaces`.
- `frontend/cli`: `interface` -> `application workflow` -> `service` -> `client integration` -> `external interfaces`.
- `frontend/web-ssr`: `server-rendered presentation` -> `server-side integration` -> `API Gateway`.
- `krakend`: `gateway contract` -> `routing/composition` -> `backend integration`.
- `traefik`: `edge entry` -> `routing` -> `middleware/policy` -> `upstream services`.
- `music-service`: `handler` -> `service` -> `repository` -> `model`, with service-level access to model objects.

Description of architectural patterns used

- **N-tier architecture:** The system is organized into five tiers: presentation, edge/access, orchestration, business services, and data/infrastructure.
- **Separated edge and orchestration layers:** Traefik and KrakenD are intentionally placed in different tiers because Traefik handles public access concerns while KrakenD handles API contract and backend routing concerns.
- **Layered style inside services:** Backend services separate handlers/controllers, application services, repositories, and models where applicable.
- **Layered event-driven processing:** Event consumers act as entry points for asynchronous workflows, while service logic and persistence remain separated.
- **Allowed-to-use dependency rule:** Higher tiers depend on lower tiers through explicit HTTP, WebSocket, AMQP, or database connectors.

4. Decomposition Structure

Decomposition View



Description of architectural elements and relations

System decomposition

- **FitBeat System**
 - **Presentation Subsystem**
 - Web CSR frontend (`frontend/web`)
 - Web SSR frontend (`frontend/web-ssr`)
 - CLI frontend (`frontend/cli`)
 - **Edge and Access Subsystem**
 - Reverse proxy (`traefik`)
 - TLS certificates and CORS middleware
 - **API Orchestration Subsystem**
 - API Gateway (`krakend`)

- Public API endpoint contracts
- Identity Subsystem
 - User service
 - Users database
 - Spotify OAuth integration
- Workout and Playback Subsystem
 - Music service
 - Music/session database
 - WebSocket player control
 - Spotify playback integration
- Gamification Subsystem
 - Achievements service
 - Achievements database
- Notification Subsystem
 - Notification service
 - Notification database
 - Email integration
- Async Processing Subsystem
 - Event processor
 - Event database
 - RabbitMQ queues/exchanges
- Shared Infrastructure Subsystem
 - Docker networks
 - RabbitMQ broker
 - Service-to-service shared secret

Main relations

- Presentation subsystem depends on the edge/access subsystem for public entry.
- Edge/access subsystem forwards API traffic to API orchestration and SSR traffic to the SSR frontend.
- API orchestration subsystem depends on backend service contracts.
- Business subsystems own their data components and do not share database schemas.
- Workout/playback publishes events consumed by gamification, notifications, and event processing.
- Notification subsystem uses an explicit internal channel to request user contact data from the identity subsystem.
- External integrations are encapsulated behind the services that need them.

Description of architectural patterns used

- **Functional decomposition by business capability:** Services are organized by domain responsibility: identity, workout/playback, gamification, notifications, and async metrics.
- **Infrastructure decomposition:** Edge, gateway, broker, networks, certificates, and databases are separated from business services.
- **Integration decomposition:** Spotify and email providers are modeled as external systems and accessed through service-owned integration logic.
- **High cohesion and loose coupling:** Each subsystem owns a focused responsibility and communicates through REST, WebSocket, AMQP, or explicit S2S contracts.

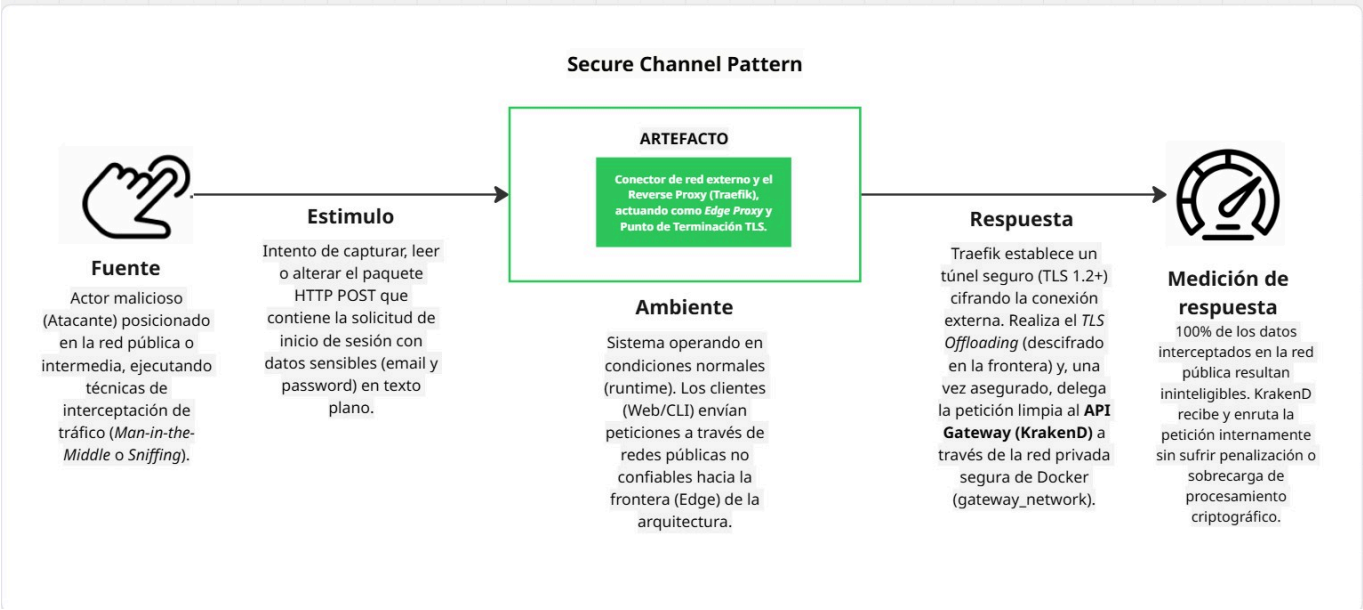
- **Database-per-service:** Data ownership follows subsystem boundaries.

Quality Attributes

Security

Security scenarios

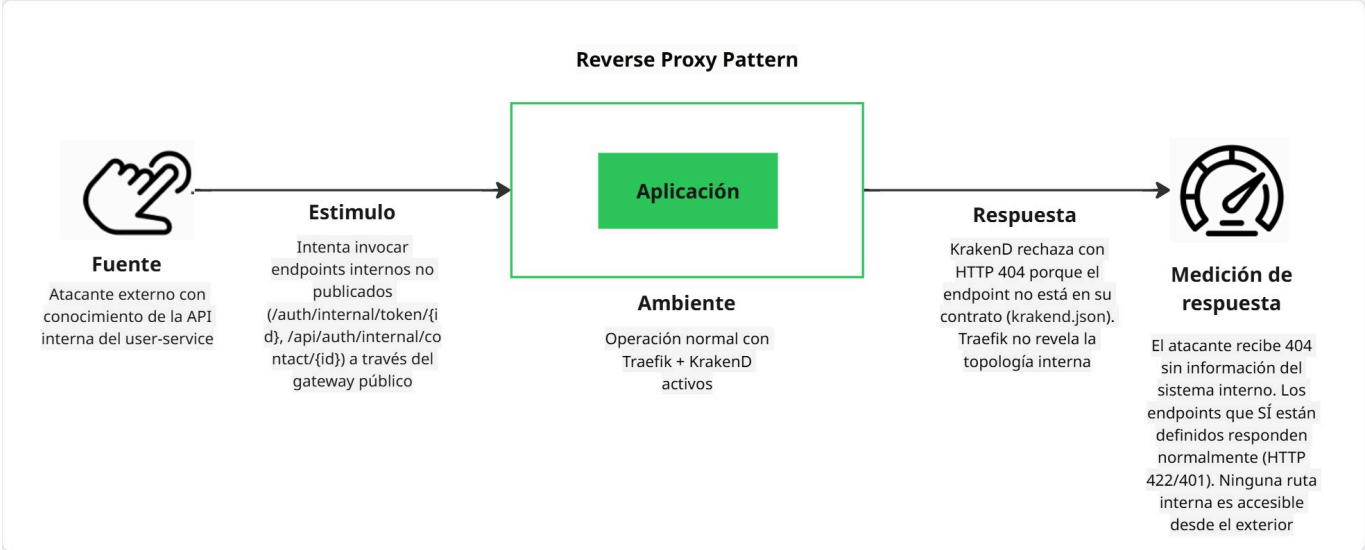
Scenario 1 - Secure Channel Pattern



Element	Description
Source	Legitimate user using the web frontend or CLI, with a potential attacker listening on the network.
Stimulus	The user sends credentials or tokens during login, registration, or authenticated requests.
Artifact	External connector between client applications and the public FitBeat entry point.
Environment	Normal operation through Traefik using HTTPS on port 443.
Response	Traefik terminates TLS 1.2+ using FitBeat certificates and forwards the request internally after the secure channel is established.
Response measure	Credentials and tokens are not readable in transit; packet capture shows TLS handshake/encrypted payload instead of plain JSON credentials.

Description: this scenario protects confidentiality and integrity for sensitive user traffic. It comes from the laboratory evidence where HTTP on port 8090 exposed login payloads in clear text, while HTTPS on port 443 encrypted the same information.

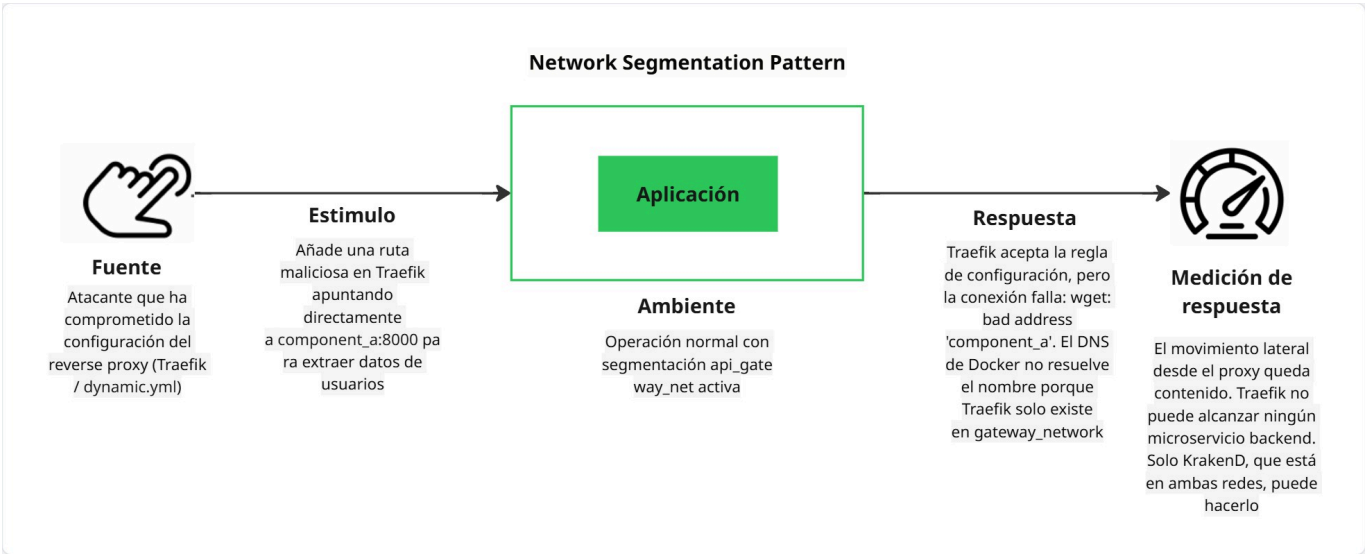
Scenario 2 - Reverse Proxy Pattern



Element	Description
Source	External attacker with knowledge of internal service route names.
Stimulus	The attacker attempts to call internal or non-public routes through the public gateway.
Artifact	Internal backend endpoints and service topology.
Environment	Normal operation with Traefik in front of KrakenD.
Response	Traefik forwards only allowed API path prefixes to KrakenD; KrakenD responds only for endpoints defined in <code>krakend.json</code> .
Response measure	Non-whitelisted routes return rejection such as <code>404 Not Found</code> , and no internal host, port, database, or service topology is revealed.

Description: Traefik is a pure reverse proxy at the edge. It knows KrakenD and the SSR frontend, but it does not know the addresses of backend microservices. KrakenD then applies the API contract whitelist.

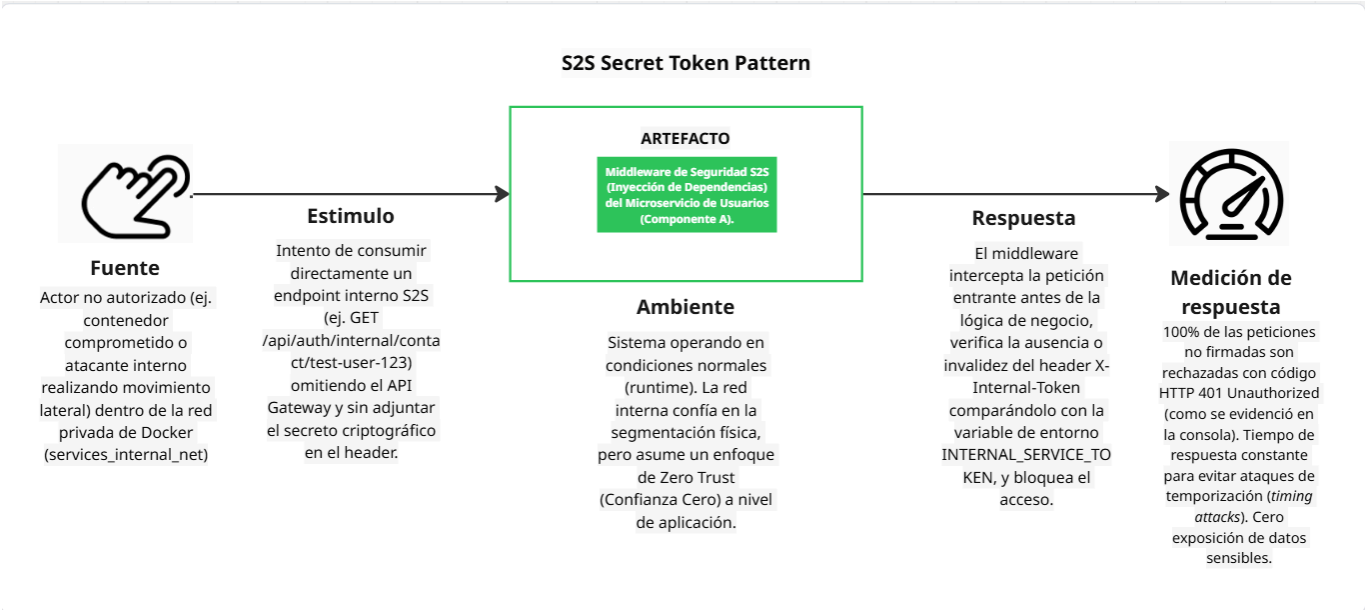
Scenario 3 - Network Segmentation Pattern



Element	Description
Source	Attacker who compromises a container such as <code>notification-service</code> or attempts lateral movement from the reverse proxy.
Stimulus	The attacker tries to reach unrelated services or databases directly.
Artifact	Docker networks, backend microservices, databases, and RabbitMQ.
Environment	Normal operation with segmented networks and service-specific memberships.
Response	Docker networking prevents name resolution and TCP connectivity unless both components share an explicit network.
Response measure	A compromised component cannot reach unrelated databases or services; for example, Traefik cannot resolve <code>component_a</code> , and <code>notification-service</code> cannot reach the users database.

Description: this pattern limits the blast radius of a compromise. The system is divided into `gateway_network`, `api_gateway_net`, service-owned database networks, `messaging_net`, and `services_internal_net`.

Scenario 4 - Secret Token Pattern for S2S communication



Element	Description
Source	Unauthorized internal actor, compromised container, or service without the internal secret.
Stimulus	The actor invokes an internal endpoint such as <code>/api/auth/internal/contact/{user_id}</code> without <code>X-Internal-Token</code> .
Artifact	Internal service-to-service endpoints in backend microservices.
Environment	Normal operation inside Docker internal networks.
Response	Middleware validates <code>X-Internal-Token</code> using timing-safe comparison and rejects missing or invalid tokens before business logic executes.

Element	Description
Response measure	Unauthorized calls return 401 Unauthorized ; authorized calls with the valid token pass middleware and reach business logic.

```

backe@Davod070105 MINGW64 ~/OneDrive/Documentos/UNAL/9 MATRICULA/PROTOTIPO 1 (JuanChacon)
$ docker run --rm -it --network prototype_2_services_internal_net curlimages/curl sh
~ $ curl -i -X GET http://fb_users_ms:8000/api/auth/internal/contact/test-user-123
HTTP/1.1 401 Unauthorized
date: Sat, 16 May 2026 21:13:29 GMT
server: uvicorn
content-length: 50
content-type: application/json

{"message":"internal token invalido","details":[]}~ $ curl -i -X GET http://fb_users_ms:8000/api/auth/internal/contact/test-user-123 \
> -H "X-Internal-Token: super_secreto_local_123"
HTTP/1.1 404 Not Found
date: Sat, 16 May 2026 21:13:39 GMT
server: uvicorn
content-length: 48
content-type: application/json

{"message":"usuario no encontrado","details":[]}~ $ exit

```

Description: network location is not treated as trust. Even if an attacker reaches an internal route, the service rejects the request unless the deployment-injected shared secret is present and valid.

Applied architectural tactics

- **Authenticate actors:** JWT authenticates external users; **X-Internal-Token** authenticates internal service-to-service calls.
- **Authorize access:** KrakenD exposes only whitelisted endpoints and internal routes require explicit authorization.
- **Limit access:** Docker networks restrict which containers can communicate.
- **Encrypt data in transit:** TLS 1.2+ secures external HTTP traffic.
- **Limit exposure:** Traefik hides internal service addresses and forwards backend API traffic only to KrakenD.
- **Detect and reject malformed access:** Gateway and middleware return controlled errors for unknown or unauthorized routes.
- **Separate privilege zones:** Public edge, API gateway, databases, broker, and S2S communication are placed in different network zones.

Applied architectural patterns

- **Secure Channel Pattern:** implemented with HTTPS/TLS termination at Traefik and FitBeat certificates.
- **Reverse Proxy Pattern:** implemented with Traefik as the edge component that hides backend topology.
- **API Gateway Pattern:** implemented with KrakenD as the explicit public API contract.
- **Network Segmentation Pattern:** implemented with Docker networks that isolate DMZ, API, data, messaging, and S2S zones.
- **Secret Token / Shared Secret Pattern:** implemented with the **FITBEAT_INTERNAL_SECRET** environment variable and **X-Internal-Token** header.
- **Defense in Depth:** combines TLS, reverse proxy routing, endpoint whitelisting, network isolation, and internal token validation.

Performance and Scalability

Performance scenarios

Scenario 1 - Real-time playback control under concurrent training sessions

Element	Description
Source	Concurrent active users in training sessions.
Stimulus	Users send WebSocket playback commands such as play, pause, next, and previous.
Artifact	<code>music-service</code> , WebSocket connector, Spotify integration, and gateway path.
Environment	Normal operation during peak training usage.
Response	The system processes playback commands concurrently and avoids blocking session creation, event publication, or UI feedback.
Response measure	Target latency for command handling is under 200 ms for normal local-network conditions, with no blocked request threads during concurrent WebSocket handling.

Scenario 2 - Asynchronous event processing without blocking training sessions

Element	Description
Source	<code>music-service</code> and other producers emitting business events.
Stimulus	A burst of training-session events is generated while users continue controlling playback.
Artifact	RabbitMQ, <code>achievements-service</code> , <code>notification-service</code> , and <code>event-processor</code> .
Environment	Normal operation with event consumers active; consumers may process at different speeds.
Response	Producers publish events to RabbitMQ and return to the user-facing workflow without waiting for all consumers to finish.
Response measure	Active training flows remain responsive while consumers process queued events asynchronously; queue depth can absorb temporary spikes.

Applied architectural tactics

- **Introduce concurrency:** The Go-based `music-service` handles WebSocket connections and playback operations concurrently.
- **Use asynchronous processing:** RabbitMQ removes achievement, notification, and metric processing from the critical user path.
- **Maintain multiple copies / horizontal scalability:** Stateless services such as `user-service`, `music-service`, `achievements-service`, and `notification-service` can be replicated behind Traefik/KrakenD in a production deployment.
- **Bound service responsibilities:** Each microservice focuses on one capability, reducing contention and allowing independent scaling.
- **Use efficient communication paths:** WebSocket is used for repeated player commands instead of creating a new HTTP request for each interaction.

- **Queue requests during bursts:** RabbitMQ buffers event traffic when consumers are temporarily slower than producers.

Applied architectural patterns

- **Broker Pattern:** RabbitMQ decouples producers from consumers and improves resilience under event bursts.
- **Load Balancing Pattern:** Traefik/KrakenD can distribute traffic across replicated stateless services.
- **Microservices Pattern:** independent services can be scaled according to domain-specific load.
- **Database-per-service Pattern:** each service can scale its persistence according to its own read/write profile.
- **WebSocket communication pattern:** persistent connections reduce overhead for real-time player control.

Performance testing analysis and results

The performance validation focuses on two critical paths:

1. **Synchronous real-time path:** client -> Traefik -> KrakenD -> **music-service** WebSocket/HTTP session endpoints.
2. **Asynchronous event path:** **music-service** -> RabbitMQ -> **achievements-service**, **notification-service**, and **event-processor**.

Expected test design:

- Use a load generator such as k6 or JMeter to simulate concurrent session creation and playback commands.
- Measure request latency, WebSocket command round-trip time, error rate, and throughput.
- Monitor RabbitMQ queue depth, consumer acknowledgement rate, and processing delay.
- Compare active-session responsiveness with and without event bursts.

Preliminary architectural analysis:

- The user-facing training flow is protected from heavy background work because achievement evaluation, notifications, and metric aggregation are moved to RabbitMQ consumers.
- The **music-service** is the most latency-sensitive component because it owns playback and WebSocket control.
- The gateway/reverse-proxy chain adds one additional hop, but it centralizes routing and security controls.
- The main scalability risks are Spotify API dependency latency, WebSocket connection volume, RabbitMQ consumer lag, and database write contention in event-heavy scenarios.

Result interpretation criteria:

- Playback command latency should remain below the defined target during normal concurrent load.
- API errors should remain close to zero for valid requests.
- RabbitMQ queues may grow during bursts, but they must drain after the burst without blocking new training sessions.

- No single consumer should become a permanent bottleneck; slow consumers can be scaled independently.